UNIVERSITY SCHOLARSHIP ALLOCATION SYSTEM

Python Programming Case Study

1. Problem Statement

Universities receive scholarship applications from many students. Manually checking aggregate marks, subject-wise minimum scores, attendance, and family income can take time and may cause errors. A systematic method is needed to identify eligible applicants, rank them fairly, and resolve ties using predefined rules.

2. Objective

To develop a Python program that collects applicant details, checks all scholarship eligibility conditions, ranks eligible students, applies tie-breaking rules, and displays the final merit list.

3. Assumptions

- Aggregate marks must be 75% or above.
- Each of the three subject marks must be at least 60.
- Attendance must be 75% or above.
- Annual family income must be INR 2,50,000 or below.
- An applicant is eligible only when all four conditions are satisfied.
- Ranking order: higher aggregate marks, higher total of the three subject marks, higher attendance, lower family income, then alphabetical order of name.

4. Algorithm

1. Start the program and read the number of applicants.
2. For each applicant, read the name, aggregate marks, three subject marks, attendance, and annual family income.
3. Check whether aggregate marks are at least 75, every subject mark is at least 60, attendance is at least 75, and income is not more than INR 2,50,000.
4. If all conditions are true, add the applicant to the eligible list; otherwise, mark the applicant as not eligible.
5. Sort eligible applicants using the predefined ranking and tie-breaking rules.
6. Display the ranked scholarship merit list and the applicants who are not eligible.
7. Stop.

5. Flowchart

START

↓

Read number of applicants

↓

Read each applicant's name, marks, attendance and income

↓

Check all eligibility conditions

↓

Eligible? YES → Add to eligible list

NO → Add to not-eligible list

↓

More applicants? YES → Read next applicant

NO → Sort eligible applicants

↓

Apply ranking and tie-breaking rules

↓

Display merit list and not-eligible list

↓

END

6. Pseudocode

```
BEGIN
    INPUT number_of_applicants
    eligible_list = empty list
    not_eligible_list = empty list

    REPEAT for each applicant
```

```
        INPUT name, aggregate, subject1, subject2, subject3
        INPUT attendance, annual_income

        IF aggregate >= 75 AND subject1 >= 60 AND subject2 >= 60
           AND subject3 >= 60 AND attendance >= 75
           AND annual_income <= 250000 THEN
               ADD applicant details to eligible_list
        ELSE
               ADD applicant name to not_eligible_list
        END IF
    END REPEAT

    SORT eligible_list BY:
        aggregate (descending),
        subject total (descending),
        attendance (descending),
        income (ascending),
        name (A to Z)

    DISPLAY ranked eligible_list
    DISPLAY not_eligible_list
END
```

7. Python Code

```
eligible = []
not_eligible = []

n = int(input("Enter number of applicants: "))

for i in range(n):
    print("\\nApplicant", i + 1)
    name = input("Enter name: ")
    aggregate = float(input("Enter aggregate marks (%): "))

    s1 = float(input("Enter subject 1 marks: "))
    s2 = float(input("Enter subject 2 marks: "))
    s3 = float(input("Enter subject 3 marks: "))

    attendance = float(input("Enter attendance (%): "))
    income = float(input("Enter annual family income (INR): "))

    subject_total = s1 + s2 + s3

    if (aggregate >= 75 and s1 >= 60 and s2 >= 60
            and s3 >= 60 and attendance >= 75
            and income <= 250000):
        eligible.append([name, aggregate, subject_total,
                         attendance, income])
    else:
        not_eligible.append(name)

# Apply ranking and tie-breaking rules
eligible.sort(key=lambda x: (-x[1], -x[2], -x[3], x[4], x[0]))

print("\\nSCHOLARSHIP MERIT LIST")
if len(eligible) == 0:
    print("No eligible applicants.")
else:
    for rank, student in enumerate(eligible, start=1):
        print(rank, student[0],
              "Aggregate:", student[1],
              "Subject Total:", student[2],
              "Attendance:", student[3],
              "Income:", student[4])

print("\\nNOT ELIGIBLE APPLICANTS")
if len(not_eligible) == 0:
    print("None")
else:
    for name in not_eligible:
        print(name)
```

8. Expected Output (Illustration)

```
SCHOLARSHIP MERIT LIST
1  Anil   Aggregate: 91   Subject Total: 270   Attendance: 96   Income: 180000
2  Bindu  Aggregate: 86   Subject Total: 255   Attendance: 92   Income: 150000

NOT ELIGIBLE APPLICANTS
Chandu
```